PROJEKT SEMESTRALNY

Zarządzanie Sklepu

Panel Administracyjny E-Commerce

Technologie: Java · Spring Boot · Hibernate · MySQL · Thymeleaf
Rok akademicki: 2026/2027

Informatyka, rok 3
Wykonał: Maksym Pavlovskyl

1. Opis projektu

ShopAdmin to webowy panel administracyjny dla sklepu internetowego. Umożliwia zarządzanie klientami, produktami i zamówieniami oraz dostarcza analitykę sprzedażową w czasie rzeczywistym. Dostęp chroniony jest logowaniem — tylko zalogowany administrator może korzystać z systemu.

Główne funkcje:

- Zarządzanie klientami – dodawanie, edycja, usuwanie, wyszukiwanie
- Zarządzanie produktami – katalog, edycja cen, uzupełnianie stanów magazynowych
- Obsługa zamówień – składanie, zmiana statusu, anulowanie ze zwrotem stanu magazynu
- Analityka – przychody, best-sellery, top klienci, wartość magazynu
- Bezpieczeństwo – Spring Security, formularz logowania z podglądem hasła

---

2. Użyte technologie

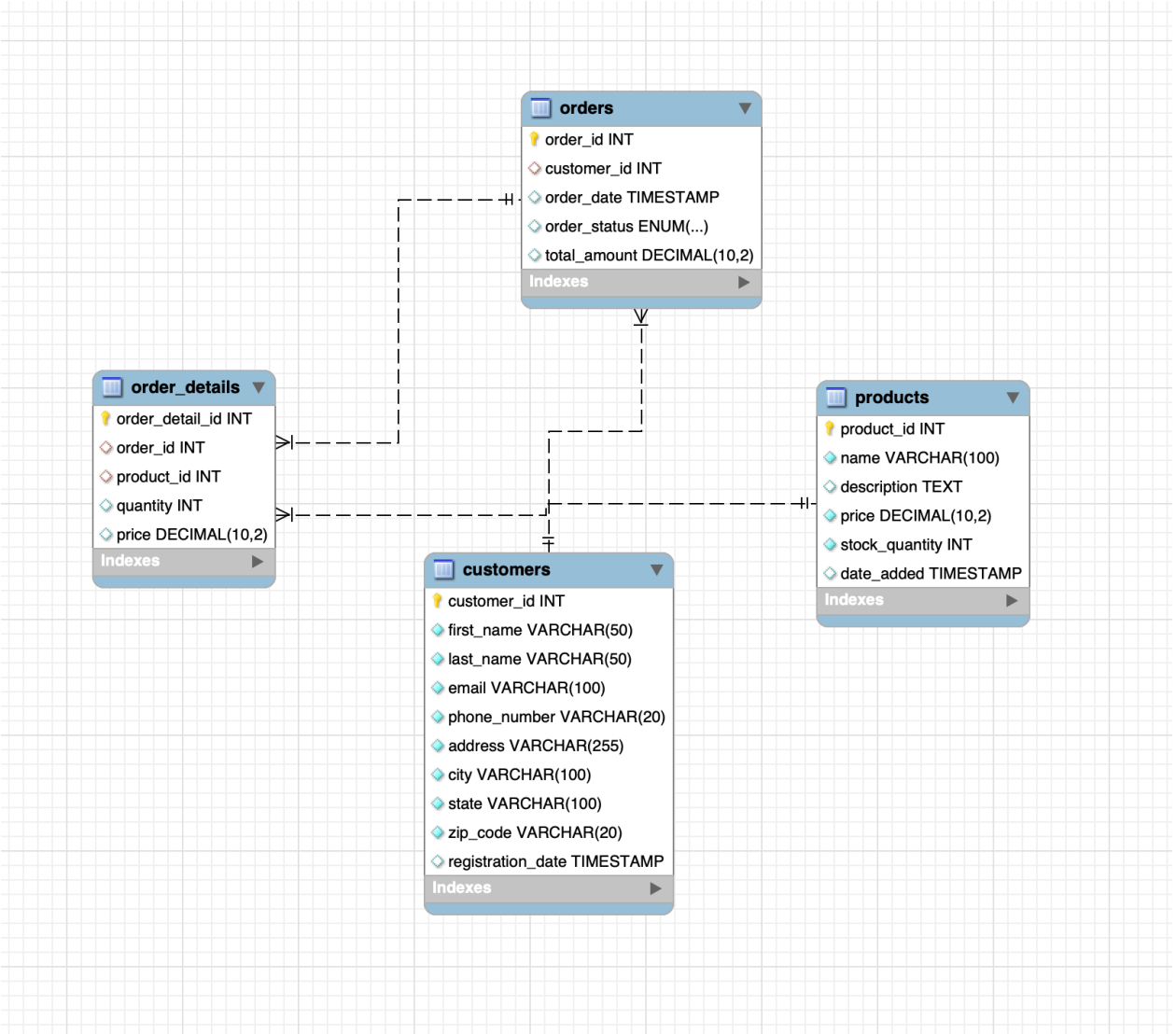
Technologia	Wersja	Zastosowanie
Java	17	Język backendowy
Spring Boot	2.7.18	Framework webowy, DI, MVC
Spring Security	2.7.x	Uwierzytelnianie i autoryzacja
Hibernate / JPA	5.6.x	ORM – mapowanie obiektowo-relacyjne
MySQL	8.x	Relacyjna baza danych
Thymeleaf	3.x	Silnik szablonów HTML po stronie serwera
Maven	3.x	Zarządzanie zależnościami i budowanie

---

3. Schemat bazy danych

Baza danych składa się z 4 tabel powiązanych kluczami obcymi w relacjach jeden-do-wielu (1:N):

- customers → orders: jeden klient może mieć wiele zamówień
- orders → order_details: jedno zamówienie zawiera wiele pozycji
- products → order_details: jeden produkt może wystąpić w wielu pozycjach zamówień



Rysunek 1. Schemat relacyjny bazy danych ecommerce_store

customers	products	orders	order_details
🔑 **customer_id (PK)**	🔑 product_id (PK)	🔑 order_id (PK)	🔑 order_detail_id (PK)
first_name	name	customer_id (FK)	order_id (FK)
last_name	description	order_date	product_id (FK)
email (UNIQUE)	price DECIMAL(10,2)	order_status ENUM	quantity
phone_number	stock_quantity	total_amount DECIMAL(10,2)	price DECIMAL(10,2)
address, city, state, zip_code	date_added TIMESTAMP		
registration_date TIMESTAMP			

Statusy zamówienia (ENUM):

- Pending – przyjęte, oczekuje na realizację
- Shipped – wysłane przez administratora
- Delivered – dostarczone do klienta
- Cancelled – anulowane (stan magazynu przywrócony)

---

4. Architektura aplikacji

Aplikacja oparta jest na wzorcu MVC (Model-View-Controller) z dodatkową warstwą DAO:

Warstwa	Technologia	Odpowiedzialność
Model	JPA @Entity	Klasy odwzorowujące tabele bazy danych
DAO	Hibernate / EntityManager	Zapytania JPQL, operacje CRUD na bazie
Controller	Spring MVC @Controller	Obsługa żądań HTTP, przekazywanie danych do widoku
View	Thymeleaf HTML	Renderowanie stron po stronie serwera
Security	Spring Security	Logowanie, wylogowanie, ochrona endpointów

---

5. Hierarchia plików projektu

Backend (Java)

```
src/main/java/org/example/
├── Main.java                          ← Punkt startowy
(@SpringBootApplication)
├── config/
│   └── SecurityConfig.java            ← Konfiguracja Spring Security
├── controller/
│   ├── AnalyticsController.java       ← /analytics - pulpit główny
│   ├── AuthController.java            ← /login
│   ├── CustomerController.java        ← /customers (CRUD)
│   ├── ProductController.java         ← /products (CRUD + restock)
│   └── OrderController.java           ← /orders (CRUD + statusy)
├── dao/
│   ├── AnalyticsDAO.java              ← Zapytania analityczne JPQL
│   ├── CustomerDAO.java               ← Operacje na klientach
│   ├── ProductDAO.java                ← Operacje na produktach
│   └── OrderDAO.java                  ← Operacje na zamówieniach
└── model/
    ├── Customer.java                  ← @Entity - tabela customers
    ├── Product.java                   ← @Entity - tabela products
    ├── Order.java                     ← @Entity - tabela orders
    └── OrderDetail.java               ← @Entity - tabela order_details
```

Frontend (Thymeleaf + CSS)

```
src/main/resources/
├── application.properties            ← Konfiguracja bazy danych i
serwera
├── static/css/admin.css              ← Globalny arkusz styli
└── templates/
    ├── login.html                    ← Strona logowania
    ├── analytics.html                ← Pulpit analityczny
    ├── customers.html                ← Lista klientów
    ├── customer-detail.html          ← Profil klienta
    ├── customer-form.html            ← Formularz klienta
    ├── products.html                 ← Lista produktów
    ├── product-form.html             ← Formularz produktu
    ├── orders.html                   ← Lista zamówień
    ├── order-detail.html             ← Szczegóły zamówienia
    ├── order-form.html               ← Formularz zamówienia
    └── fragments/layout.html         ← Wspólna nawigacja boczna
```

---

6. Kluczowe fragmenty kodu

6.1 Mapowanie encji – Customer.java

@Entity rejestruje klasę w Hibernate. @Id oznacza klucz główny. @OneToMany definiuje relację 1:N do tabeli orders.

```
@Entity @Table(name = "customers")
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "customer_id")
    private int customerId;
    @Column(name = "registration_date")
    @Temporal(TemporalType.TIMESTAMP)
    private Date registrationDate;
    @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
    private List<Order> orders;
}
```

6.2 Zapytanie analityczne – AnalyticsDAO.java

JPQL z agregacją SUM i GROUP BY zwracający top klientów według przychodu.

```
public List<Object[]> getTopCustomers(int limit) {
    return em.createQuery(
        "SELECT c.firstName, c.lastName, SUM(o.totalAmount), COUNT(o) " +
        "FROM ShopOrder o JOIN o.customer c " +
        "WHERE o.orderStatus <> :cancelled " +
        "GROUP BY c.customerId ORDER BY SUM(o.totalAmount) DESC",
        Object[].class)
    .setParameter("cancelled", OrderStatus.Cancelled)
    .setMaxResults(limit).getResultList();
}
```

6.3 Składanie zamówienia – OrderDAO.java

Walidacja stanu magazynu dla każdej pozycji przed zapisem, atomowe odjęcie zapasów.

```
@Transactional
public Order placeOrder(Customer customer, List<OrderDetail> items) {
    for (OrderDetail od : items) {
        Product p = em.find(Product.class,
od.getProduct().getProductId());
        if (p.getStockQuantity() < od.getQuantity())
            throw new RuntimeException("Brak towaru: " + p.getName());
    }
    Order order = new Order(managedCustomer, total);
    order.setOrderDate(new Date());
    em.persist(order);
    for (OrderDetail od : items) { em.persist(od);
        p.setStockQuantity(p.getStockQuantity() - od.getQuantity()); }
    return order;
}
```

6.4 Spring Security – SecurityConfig.java

Ochrona wszystkich endpointów poza /css/**. Hasła szyfrowane BCrypt.

```
http.authorizeRequests()
    .antMatchers("/css/**").permitAll()
    .anyRequest().authenticated()
.and().formLogin()
    .loginPage("/login").defaultSuccessUrl("/analytics", true)
.and().logout().logoutSuccessUrl("/login?logout");
```

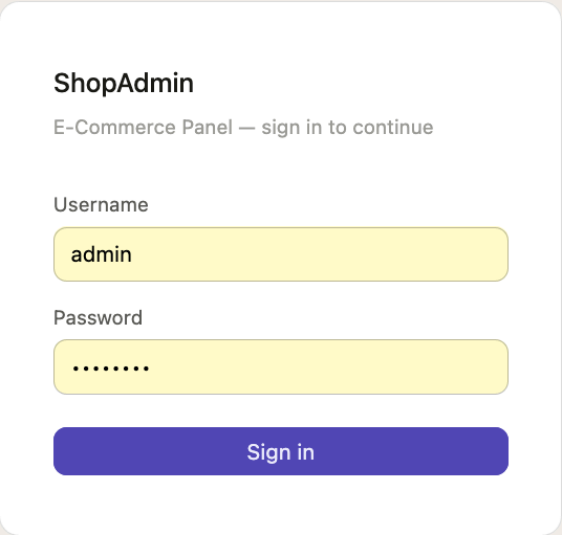
---

7. Wygląd aplikacji

Interfejs zbudowany jest w oparciu o własny system CSS bez zewnętrznych frameworków UI. Układ dzieli się na stały sidebar nawigacyjny (lewy panel) oraz główną strefę treści z topbarem i zawartością strony.

7.1 Strona logowania

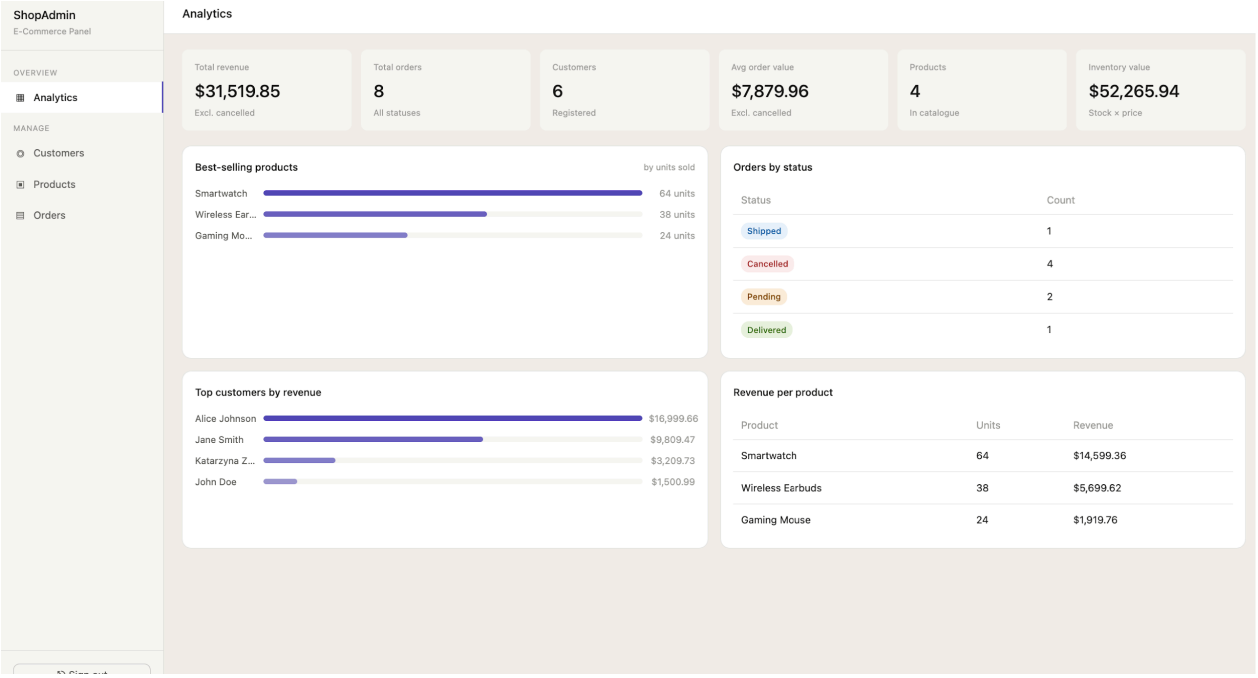
Prosta karta logowania wyśrodkowana na ekranie. Zawiera pola nazwy użytkownika i hasła z przyciskiem podglądu (ikona oka) oraz przycisk Sign in. Komunikat błędu pojawia się przy niepoprawnych danych; komunikat potwierdzenia po wylogowaniu.



Rysunek 2. Strona logowania – /login

7.2 Pulpit analityczny

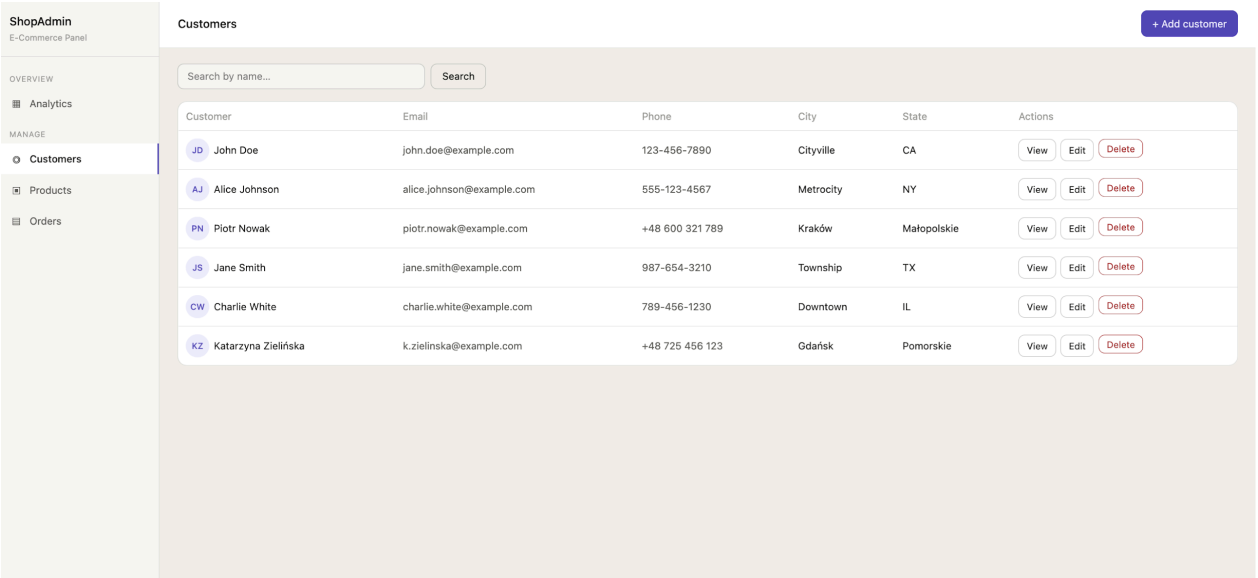
Ekran główny po zalogowaniu. Górna część zawiera 6 kart KPI: łączny przychód, liczba zamówień, klienci, średnia wartość zamówienia, liczba produktów, wartość magazynu. Poniżej dwa panele: wykres słupkowy najlepiej sprzedających się produktów (wg jednostek) oraz podział zamówień według statusu z kolorowymi etykietami. Dolna część to ranking top klientów według przychodu i tabela przychodów na produkt.



Rysunek 3. Pulpit analityczny – /analytics

7.3 Lista klientów

Tabela wszystkich klientów z wyszukiwarką po nazwisku lub imieniu. Każdy wiersz zawiera awatar z inicjałami, email, telefon, miasto, stan oraz przyciski akcji: View (profil), Edit (edycja), Delete (usunięcie z potwierdzeniem). Przycisk '+ Add customer' w topbarze otwiera formularz dodawania.

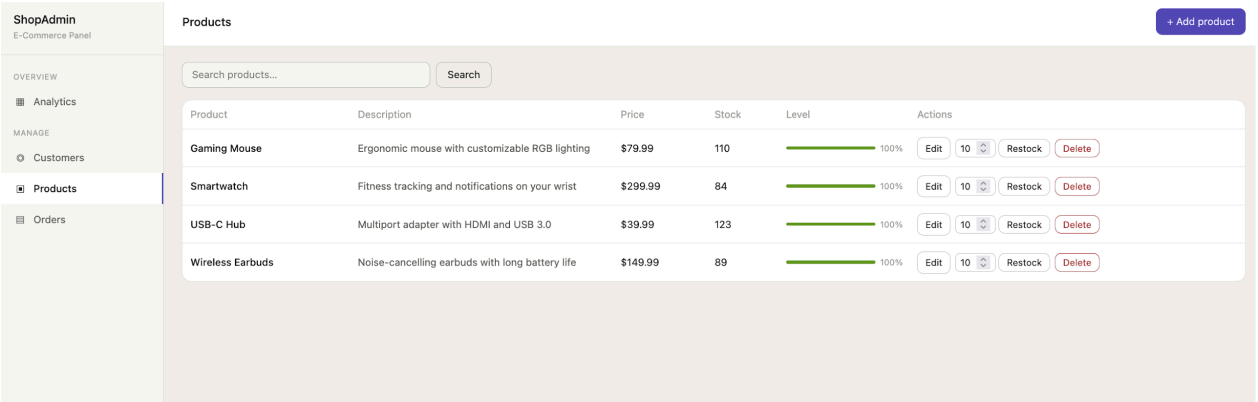


Rysunek 4. Lista klientów – /customers

7.4 Lista produktów

Tabela produktów z wyszukiwarką, kolumną opisu, ceną i stanem magazynowym. Kolumna Level przedstawia poziom zapasów jako pasek postępu — zielony (powyżej 50%),

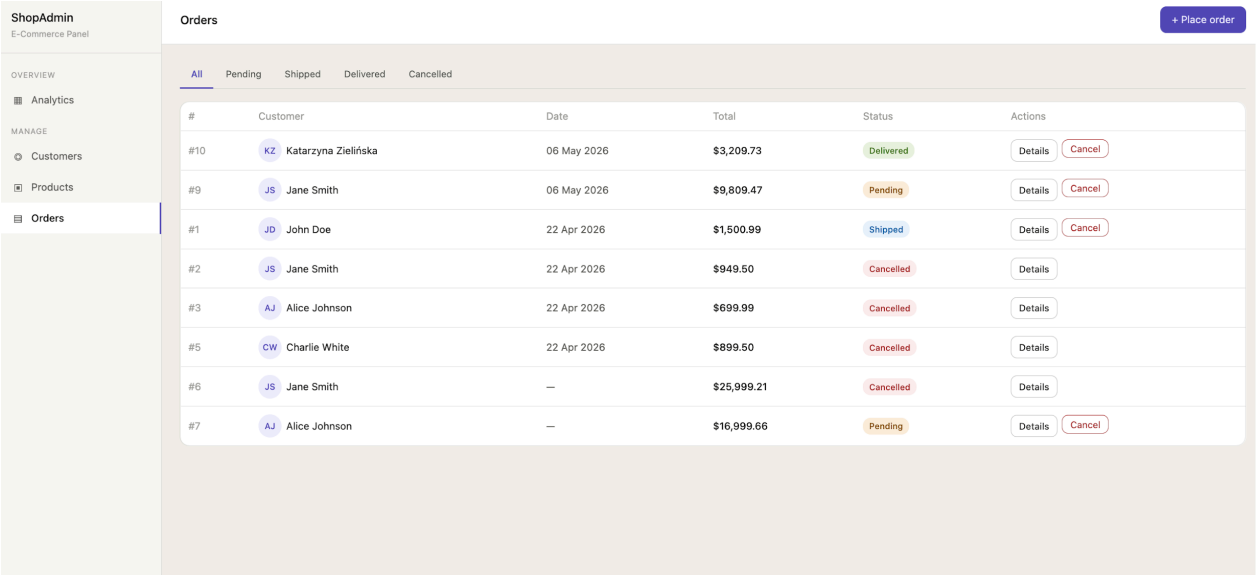
pomarańczowy (20–50%), czerwony (poniżej 20%). Inline restock: pole numeryczne i przycisk Restock bezpośrednio w wierszu. Baner czerwony pojawia się gdy produkty mają niski stan. Usunięcie produktu z historią zamówień jest blokowane.



Rysunek 5. Lista produktów – /products

7.5 Lista zamówień

Tabela zamówień z zakładkami filtrowania: All, Pending, Shipped, Delivered, Cancelled. Każdy wiersz zawiera numer zamówienia, klienta (awatar + imię), datę, łączną kwotę, kolorową etykietę statusu oraz przyciski Details i Cancel. Anulowanie zamówienia automatycznie przywraca stany magazynowe. Przycisk '+ Place order' otwiera formularz składania zamówienia dla wybranego klienta.



Rysunek 6. Lista zamówień – /orders

---

8. Instrukcja uruchomienia

Wymagania

- Java 17+ · Maven 3.x · MySQL 8.x

Kroki

- Zaimportuj bazę danych:

```
mysql -u root -p < E-Commerce.sql
```

- Ustaw hasło w application.properties:

```
spring.datasource.password=TWOJE_HASLO
spring.jpa.hibernate.ddl-auto=none
```

- Uruchom aplikację:

```
mvn spring-boot:run
```

- Otwórz: http://localhost:8080 → login: admin / hasło: admin123

---

9. Podsumowanie

Projekt ShopAdmin realizuje kompletny system zarządzania sklepem internetowym po stronie administratora. Zastosowane technologie stanowią reprezentatywny zestaw narzędzi używanych w profesjonalnych aplikacjach webowych opartych na platformie Java.

Projekt demonstruje:

- Mapowanie obiektowo-relacyjne (ORM) – adnotacje JPA/Hibernate
- Wzorzec MVC z wydzieloną warstwą DAO
- Zarządzanie transakcjami (@Transactional, readOnly)
- Integralność referencyjna – kaskadowe usuwanie z zachowaniem historii zamówień
- Uwierzytelnianie z szyfrowaniem haseł BCrypt
- Zapytania analityczne JPQL z agregacją (SUM, AVG, COUNT, GROUP BY)